

RESEARCH

Open Access



Shadowkube: enhancing Kubernetes security with behavioral monitoring and honeypot integration

Qingwang Chen^{1,2}, Yuling Liu^{1,2}, Ru Tan^{1,2}, Ze Jin^{1,2*} , Juxin Xiao^{1,2}, Xiao Wang^{1,2}, Fangjiao Zhang^{1,2} and Qixu Liu^{1,2}

Abstract

As cloud-native technologies continue to evolve, containerization and orchestration have become fundamental for deploying microservices. However, this advancement introduces significant security vulnerabilities, particularly due to vulnerabilities and misconfigurations that grant attackers excessive control over clusters. Existing works, including model-based learning and static rule-based approaches, suffer from limitations such as false positives and maintenance overhead, which pose significant challenges to cloud-native security.

To mitigate intrusion targeting container orchestration, we present *ShadowKube*, an innovative active defense framework tailored for Kubernetes. *ShadowKube* integrates behavioral monitoring with shadow honeypots to effectively detect and neutralize anomalous behavior. By establishing behavioral baselines to identify deviations and converting compromised nodes into honeypots, *ShadowKube* isolates and traps attackers, thereby mitigating the threats they pose. Comprehensive evaluations demonstrate *ShadowKube*'s ability to detect and migrate exploitations across 43 severe CVEs and 7 common misconfiguration types. Deployment in a live environment further validates its effectiveness, with *ShadowKube* identifying 635 attack attempts, successfully decoying 23 active attacks. Additionally, *ShadowKube* could isolate attackers and convert affected nodes into honeypots within seconds. These results highlight *ShadowKube*'s efficacy as a robust solution for enhancing security in Kubernetes clusters, offering a proactive defense mechanism against both current and emerging threats.

Keywords Kubernetes, Honeypot, Behavioral monitor, Cloud native security

Introduction

The rapid expansion of cloud-native applications has significantly propelled the use of container and orchestration technologies, capitalizing on their agility, scalability, and service-oriented architecture to address various

challenges encountered during the business development lifecycle. Container technologies like Docker (2024a) and Podman (2024) are widely adopted for the distribution and deployment of applications. For managing these extensive container deployments, technologies such as Docker Swarm (2024b), OpenShift (2024), and especially Kubernetes (2024b) have been implemented. Kubernetes has become the leading orchestration platform, distinguished by its comprehensive automation, scalability, and reliability, thereby setting the benchmark for the deployment of modern cloud-native applications. Reports from several organizations have documented significant

*Correspondence:

Ze Jin
jinze@iie.ac.cn

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China

adoption of Kubernetes for container orchestration within production environments (Redhat 2023a; Canonical 2024; Redhat 2023b; VMware 2023; thenewstack 2023; spectrocloud 2023; Wiz 2024).

Kubernetes Security Vulnerabilities. The surging popularity of Kubernetes and containerized applications has unveiled numerous security vulnerabilities. The shift towards containerized deployments has revealed new potential attack vectors, as the intricate and dynamic nature of Kubernetes architecture amplifies the possibility of human errors in configurations. This increase in complexity raises the risk of security issues such as container escapes (Aquasec 2024), privilege escalations (wikipedia 2024b), and insufficient network policies (Kubernetes 2024d). These security concerns have been highlighted by incidents that have caused development delays and financial losses from security breaches, including widespread Kubernetes mining attacks and exploits that take advantage of the RBAC (Role-based access control) system to take over clusters (Microsoft 2024a; Katchinskiy and Morag 2023).

In these incidents, attackers typically exploit cluster misconfigurations or vulnerabilities within cluster applications to obtain elevated privileges, such as the ability to create containers. With these privileges, they deploy numerous malicious resources under their control, including containers embedded with backdoors or cryptocurrency mining malware. The inherently distributed architecture of Kubernetes further exacerbates the situation, allowing these malicious resources to rapidly propagate across all nodes in the cluster without additional constraints. This widespread dissemination can result in significant damage to the entire cluster.

Why Imperative for Behavioral Monitoring. As highlighted in the current OWASP Top 10 (2024), compromised access control represents the foremost security challenge today, with leading cloud computing platforms like AWS (2024) and Google Cloud (2024) employing access control as a crucial defensive mechanism. In this context, an attacker exploiting a misconfiguration in Kubernetes' access control could potentially seize control of the entire cluster by leveraging roles granted overly permissive permissions. For instance, a role named "developer-role" within the "dev-namespace" may erroneously confer cluster-wide edit privileges. An attacker could exploit this by creating a service account linked to this role, thereby obtaining elevated privileges and deploying malicious pods that compromise both the host system and the cluster. Despite access control systems serving as foundational security measures for container clusters, their inherent nature makes such attacks challenging to detect. Given that this form of attack bypasses direct payload delivery to the container pod by targeting

the cluster's API server (2024a), network-level security measures find it difficult to provide adequate protection against such vulnerabilities. Consequently, monitoring container behavior emerges as an essential security mechanism supplementary to access control to counteract these threats.

Challenges in Monitoring Cluster Behavior. However, tracking container usage behavior poses significant challenges. The effectiveness challenge arises in detecting actions that indicate an attacker's control over a container cluster, especially if the attacker avoids performing clearly abnormal operations. Existing methods, such as rule-based tools (Falco 2024; KubeArmor 2024; Tetragon 2024; Aquasecurity 2024) and machine learning models (Karn et al. 2020; Tien et al. 2019; Lin et al. 2020), suffer from limitations in dynamic Kubernetes environments. For example, machine learning-based detection methods can exhibit high false positive rates due to sudden bursts of business traffic, which are common in Kubernetes clusters. Rule-based monitoring can be bypassed using techniques like symbolic links, rendering static rules ineffective. The efficiency challenge relates to the timeliness of monitoring. Delayed detection and response allows attackers to advance their strategies, such as leveraging a compromised container to extract credentials for lateral movement (wikipedia 2024a) within the cloud infrastructure. Currently, there are no effective automated mitigation solutions specifically targeting Kubernetes security threats. Immediate termination of abnormal containers can also lead to resource depletion and reliance on manual intervention, undermining cluster availability and placing high demands on human response speed. Therefore, it is essential to implement an automated, real-time behavior monitoring mechanism that overcomes these limitations and swiftly identifies signs of unauthorized control.

While recent studies (Microsoft 2024b; Falco 2024; Tetragon 2024; Aquasecurity 2024; Spahn et al. 2023) have contributed to enhancing Kubernetes security, they primarily focus on monitoring and preventing anomalous behaviors through the application of static policies. This method necessitates extensive rule maintenance and remains susceptible to circumvention by attackers (Blackberry 2024). Furthermore, although Karn et al. (2020) proposes a machine learning-based approach for identifying anomalous pods specifically for cryptomining detection, none of these efforts effectively address Kubernetes security through dynamic behavioral monitoring.

To this end, this paper employs behavioral baselines for identifying anomalies in Kubernetes clusters and utilizes honeypot technology to deceive attackers. Inspired by Moving Target Defense (MTD) and the concept of shadow honeypot, we present *ShadowKube*. This system

discreetly transforms potentially compromised pods and nodes into honeypots, effectively separating attackers from the genuine production environment. We also addressed a notable technical challenge that ensuring the seamless transition of a normal node to a honeypot, aiming to migrate the attacker's activities without detection. By strategically deploying honeypots, we endeavor to disorient and capture attackers, thereby offering an innovative solution to reduce the disparity between offensive and defensive tactics in cloud security. To our knowledge, this is the first effort to use honeypots for safeguarding the Kubernetes platform and container applications.

In summary, this paper makes the following contributions:

- **Algorithm Design.** We developed a baseline algorithm leveraging the Longest Common Subsequence (LCS) and Levenshtein distance to establish normative benign behaviors of containers and to quantify anomalies in container runtime.
- **System Implementation.** We implemented *ShadowKube*, an active defense framework that uses shadow honeypots for Kubernetes security. It converts abnormal containers and hosts into honeypots without the attacker's knowledge, facilitating entrapment and defense.
- **Comprehensive Evaluation.** We conducted experiments to test *ShadowKube's* ability to detect anomalies, trap efficiency, and assess its performance impact. The system proved capable of quickly identifying and isolating abnormal behaviors in the cluster with minimal overhead. It connects affected areas to a shadow cluster for effective defense. During a one-month public network deployment, *ShadowKube* attracted 635 real attacks.

Background

Container security

In contemporary software deployment, containers have emerged as a pivotal technology, offering lightweight and portable solutions for bundling applications along with their dependencies. This approach ensures consistent execution across diverse computing environments. The core advantage of containers is their ability to share the host operating system's kernel, in contrast to traditional virtual machines, which require separate operating systems for each instance. This characteristic significantly enhances efficiency and reduces resource consumption, representing a shift towards more streamlined computing practices.

Docker (2024a), recognized as a prominent container engine, exemplifies the practical application of container technology within the cloud-native landscape. It facilitates the creation and management of containers and their images by leveraging the Linux kernel's isolation mechanisms, such as namespaces and cgroups. Docker mediates the interaction between containers and the operating system, striking a balance between isolation and performance efficiency similar to that of native applications.

However, the operational model of Docker, which requires elevated privileges to leverage specific Linux kernel functionalities, introduces notable security concerns. These vulnerabilities potentially enlarge the system's attack surface, as exemplified by issues such as CVE-2020-15257 (2020), where the exploitation of the host network by a container could lead to escape and denial-of-service attacks. Furthermore, the misconfiguration of Docker components exacerbates these risks, permitting unauthorized API access that attackers can exploit to deploy malicious containers remotely. This can facilitate a range of malicious activities, from cryptocurrency mining to ransomware deployment.

Additionally, within real-world production settings, Docker containers may be subject to unsafe configurations due to operational demands or oversight. These lapses can undermine container security isolation mechanisms, affording attackers opportunities to perform hazardous operations, or even escape to the host. Such scenarios highlight the potential for exploiting container vulnerabilities to achieve broader system access, underscoring the importance of vigilant configuration and management practices to safeguard against these threats.

Kubernetes security

Kubernetes is an open-source container orchestration platform that creates and manages containers at scale. The architecture of a Kubernetes cluster is shown in Fig. 1, which consists of a control plane(master node) and a data plane(worker nodes). Control plane holds several critical components that together for managing the state of a Kubernetes cluster and ensuring the cluster operates as expected, such as kube-apiserver, kube-scheduler, etcd, and controller-manager. Data plane is responsible for carrying out commands from the control plane and running workflows, with several components and pods running on the work nodes of the cluster. Each pod represents a single instance of a running process in cluster and can contain one or more containers that share network and storage resources. Kubernetes processes user-submitted declarative manifests to autonomously orchestrate container deployment, scaling and lifecycle management by parsing the abstract resources like Deployment, which

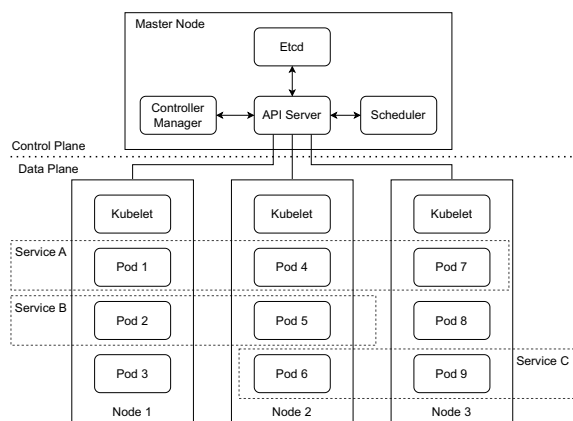


Fig. 1 Architecture of a Kubernetes cluster

leads to the transient characteristics of pods and frequent changes in pods' ip address. To overcome this, Kubernetes provide Service as a method for exposing a stable and persistent access point for a set of pods regardless of their lifecycle (creation, deletion, scaling).

The sophisticated architecture of Kubernetes, while offering extensive functionalities, also harbors a range of security vulnerabilities. These encompass threats to components, such as API servers and etcd storage, risks stemming from improper pod configurations like excessive RBAC (Role-based access control) privileges (Kubernetes 2024f), where attackers who gain access to affected pods can leverage their credentials representing roles with excessive privileges, and the attack surface presented by insecure application code. These defects can expose sensitive data, create unauthorized access points, and expand the attack surface through excessive container permissions or flawed network strategies. Notably, Tesla's Kubernetes cluster fell victim to a cryptojacking attack in 2018 due to an unauthorized dashboard configuration (Redlock 2024). Red Hat's survey reveals that over half of the respondents express concerns over cluster misconfigurations and container image vulnerabilities (Redhat 2023a).

Motivation and threat model

The widespread adoption of Kubernetes for container orchestration has significantly enhanced the scalability and efficiency of cloud-native applications. However, this increased adoption has also made Kubernetes clusters prime targets for attackers aiming to exploit vulnerabilities for malicious purposes. Existing security measures often fall short in addressing the dynamic and complex nature of Kubernetes environments. In this section, we present motivating examples that highlight the

limitations of current security strategies and introduce our threat model.

Motivation examples

Currently, two primary strategies exist for safeguarding Kubernetes clusters against malicious exploits. The first, widely adopted strategy involves the imposition of rule-based policies to restrict cluster behaviors (Falco 2024; KubeArmor 2024; Tetragon 2024; Aquasecurity 2024). This approach necessitates regular updates to the rules as container applications evolve within a dynamic cluster ecosystem. The second strategy models cluster behaviors using machine learning and deep learning techniques (Karn et al. 2020; Tien et al. 2019). While innovative, these methods have shortcomings in terms of response speed, and the high dynamicity of cluster environments greatly increases their false positive rates. Upon detecting an anomaly, the typical immediate mitigation method is to delete the container, which may have certain negative impacts on the protection of the cluster. We present three motivation examples based on the problems of these methods in Kubernetes environments to pave the way for the presentation of our methodology.

(1) Limitations of Machine Learning-Based Detection

Existing machine learning and deep learning methods for container behavior analysis widely utilize system calls as a dataset. For example, CDL (Lin et al. 2020) uses system call frequency and an autoencoder for training, which has shown promising results in experiments. However, in Kubernetes environments, sudden bursts of business traffic are common, causing system call frequencies to spike and deviate from the normal frequency ranges defined by pre-trained models. When we refactor system calls of a Flask application from Vulhub (2024) with autoencoder, the false positive rate increases from 4.9% to 27.1% due to the increased frequency of normal workflows, which is a common scenario in a Kubernetes environment. The corresponding experimental data is presented in our website (Chen 2024).

(2) Bypassing Static Rule-Based Monitoring

Falco, a well-known monitoring software for container runtimes, utilizes static rules to monitor specific behavioral characteristics within containers. However, attackers can bypass the behavior patterns defined in public rules with techniques like symbolic links. For instance, the static Falco rules illustrated in Fig. 2 trigger an alert whenever any process attempts to read a file named /

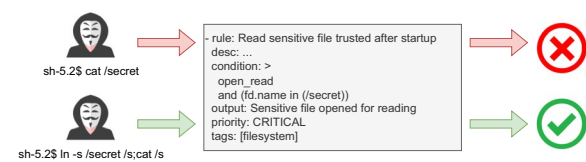


Fig. 2 Falco bypass example

secrets. Consequently, the attacker's direct attempt to read the file (depicted by the red arrow) is detected. However, the attacker can circumvent the file name restrictions enforced by the static rules by creating a symbolic link, */s*, that points to */secrets*, and then accessing the secret through the symlink. This approach allows the attacker to gain access to sensitive data (depicted by the green arrow) without triggering the alert.

(3) Ineffectiveness of Immediate Termination Strategies

In current defense strategies, directly destroying abnormal containers will result in their recreation due to Kubernetes' fault recovery mechanism. Once attackers notice the termination of active environment, they can initiate another attack. The frequent destruction and creation of containers quickly depletes cluster resources and undermines availability. As a result, handling abnormal containers heavily relies on manual analysis and intervention, which places high demands on the speed of human response.

Therefore, cloud-native defense requires an automated, real-time protection mechanism that does not rely on manually defined static rules. To address this, we introduce *ShadowKube*, which leverages behavioral baselines for attack detection and uses honeypot technology to redirect potential threats to a shadow cluster. This method isolates attackers from the production environment without disrupting their activity space, thereby limiting potential damage and achieving proactive defense and deception.

Threat model

Given Docker's prominence as the leading container engine, we construct a hypothetical threat model focused on the security of Kubernetes clusters that utilize Docker as their container runtime. This model concentrates on Kubernetes clusters deployed in production environments, which we refer to as the *external adversary framework* in production clusters. We assume that the attacker is external, possessing no insider access or prior permissions within the Kubernetes ecosystem, and is thus limited to external attack vectors.

To define the parameters of the protected cluster, we advance three hypotheses based on prevalent Kubernetes usage scenarios:

(1) Regular Kernel Updates Limit Kernel-Level Vulnerabilities

We posit that Kubernetes nodes undergo regular kernel updates, thereby mitigating kernel-related escape vulnerabilities. This assumption confines the exploitable weaknesses to application-level flaws and insecure configurations within the cluster.

(2) Pods Are Stateless and Do Not Retain Critical Data

We assert that pods do not retain data critical to the availability of services, such as user-generated data (Kubernetes 2024e). Due to the stateless nature of pods and their design to avoid reliance on external storage resources, any data they contain is transient and purged upon the termination of their lifecycle. Consequently, data essential to service availability predominantly resides within the cluster's persistent storage resources, such as PersistentVolumes.

(3) Prevalence of the One-Container-Per-Pod Model

Recognizing that the "one-container-per-pod" model is the most common Kubernetes use case (Kubernetes 2024c), we simplify our analysis by assuming each pod contains only a single workload-related container, in addition to the pause container. The pause container, which provides namespace-sharing functionality, performs no application-specific workload and is unlikely to be a target for attacks. Therefore, we treat the behavior of a pod as synonymous with that of its primary container, excluding the pause container. When attackers target a pod, only the primary container within it is affected.

We envision an attacker capable of executing a spectrum of malicious actions detailed in the Kubernetes threat matrix (Microsoft 2021). The ultimate goal of the attacker is to infiltrate the Kubernetes cluster to seize control for malicious purposes, such as cryptocurrency mining or exploiting the cluster's resources for botnet activities. Upon gaining entry, the attacker is likely to perform reconnaissance and lateral movements to expand their foothold. Potential tactics include exploiting configuration vulnerabilities, stealing credentials, and leveraging flaws in internal services (Box 2024). Furthermore, the attacker is expected to be vigilant against any system-triggered obstructions during the intrusion process.

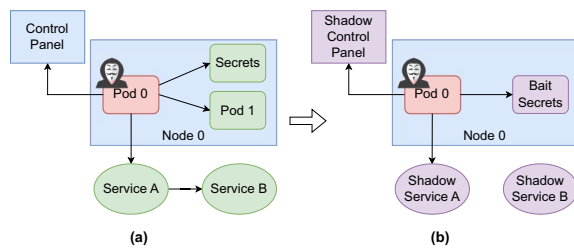


Fig. 3 An example of trapping attacker. **a** After triggering defense mechanisms, adversaries can infiltrate nothing but decoys provided by shadow cluster. **b** showcasing the efficacy of our proposed defense strategies.

The objective of *ShadowKube* is to redirect and confine the attacker's activities to a shadow cluster by employing behavioral baselines and moving target defense strategies. This approach aims to minimize disruption to the attacker's operations to prevent compromising the cluster's overall availability. By subtly guiding the attacker's behavior without raising suspicion, we aim to protect the Kubernetes cluster while simultaneously entrapping the attacker.

Design and implementation of ShadowKube

System overview

To enhance the automatic detection of anomalous behaviors, our approach involves the automatic collection of benign behavioral patterns exhibited by pods during normal workflows. Subsequently, we establish behavioral models for various categories, forming a behavioral baseline for the cluster. During the process of online

detection, suspicious behaviors are assessed by measuring their deviation from established baseline to determine their anomalous nature. Furthermore, as part of our design strategy for addressing detected anomalies, we employ honeypot technology and dynamic networking tactics to autonomously isolate and ensnare potential attackers. This approach not only mitigates the threat but also affords the defense mechanism an extended reaction timeframe. Figure 3 illustrates the successful redirection and entrapment of an attacker within this simulated environment. When attackers exploit vulnerabilities to gain control over a specific pod, they can pose a threat to the control panel, as well as to the containers, data on the same node, and other services within the cluster, as illustrated in Fig. 3 (a). After triggering defense mechanisms, adversaries can infiltrate nothing but decoys provided by shadow cluster, as shown in Fig. 3 (b), showcasing the efficacy of our proposed defense strategies.

The system comprises five components: traffic proxy, probes, detector, actuator, and shadow cluster, collectively forming the architecture and workflow depicted in Fig. 4.

ShadowKube operates through three distinct phases. Initially, it establishes a behavioral baseline by analyzing cluster activities to create a reference model of typical behavior, which is then utilized for the swift identification of anomalous behaviors. In the detection phase, *ShadowKube* closely monitors the production environment, pinpointing deviations from the baseline as potential anomalies. Upon identifying an anomaly,

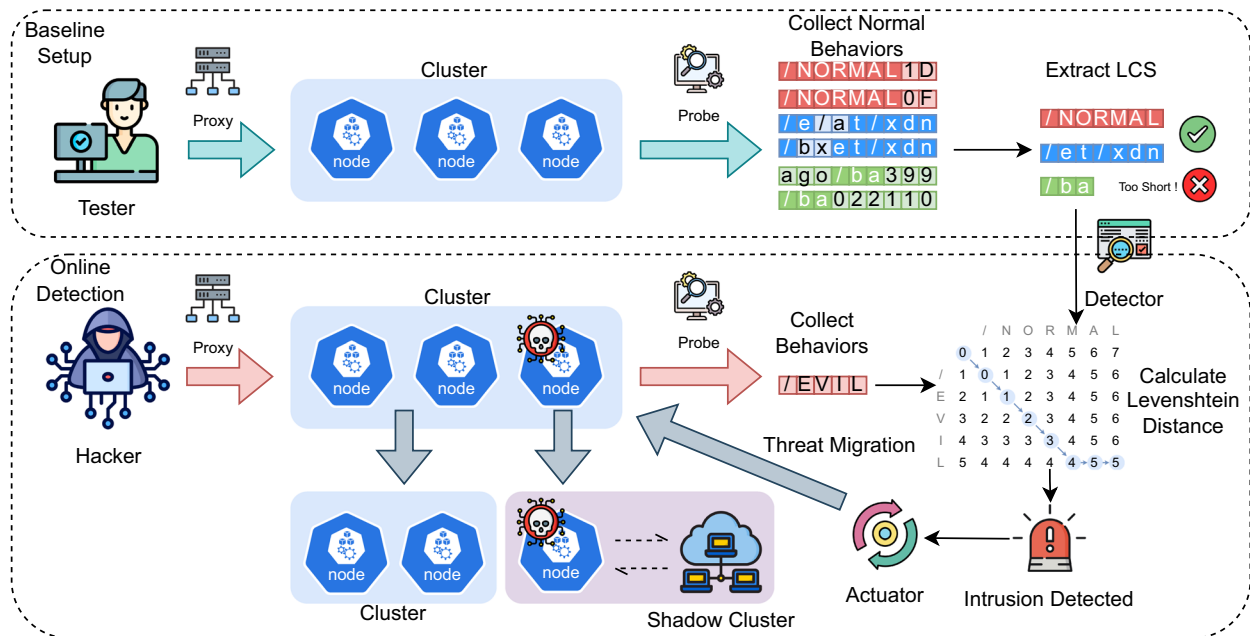


Fig. 4 Architecture of ShadowKube

Table 1 Metadata in behavioral information

Parameter	Description
Name	Unique name assigned to the pod within its namespace.
Namespace	Scope within which the pod exists.
Labels	Key-value pairs attached to the pod for identification.
Annotations	Additional metadata attached to the pod in the form of key-value pairs.
Controlled By	Resource that controls the lifecycle of the pod.

the actuator, which is responsible for formulating and executing defense strategy based on the current state of the cluster, either convert the compromised pod and node into a honeypot within an isolated shadow cluster or remove the malicious container entirely. Additionally, *ShadowKube* employs a traffic proxy to intelligently reroute overtly malicious traffic from the public network to the shadow cluster, while guiding legitimate traffic to the production environment.

Baseline setup

ShadowKube precisely categorizes the operational behaviors within Kubernetes clusters by establishing a behavioral baseline. This process involves collecting typical container behaviors, grouping pods based on their activity patterns and labels, and documenting the normal operations within the ecosystem. This baseline serves as a reference point facilitating the identification of anomalies by highlighting deviations from established norms. It emphasizes the adaptability required in the dynamic nature of containerized environments, enabling effective anomaly detection.

Standard behavior collection

To efficiently establish a baseline for standard cluster behavior, *ShadowKube* deploys probes across all nodes to monitor pod activities, including file accesses, network connections, and command executions. This monitoring provides a detailed picture of pod operations. Given the dynamic nature of the pod set, normal activities may not always occur in a specific pod but rather in its replicas. *ShadowKube* addresses this by classifying behaviors into different groups based on their metadata, as detailed in Table 1. This stratification is critical for accurately identifying anomalies and ensuring the system's adaptability to the inherent dynamism of containerized environments.

Baseline generation

In the process of establishing behavioral baselines, *ShadowKube* aims to define the standard operations of each

group within the dynamic, cloud-native environment. Acknowledging the **effectiveness challenge**, *ShadowKube* meticulously documents both the types of operations and their parameters. This approach enables the creation of comprehensive baselines by extracting feature sequences from standard pod activities, thereby enhancing the precision of behavior anomaly detection.

The methodology for deriving feature sequences is articulated in Eq. (1). To isolate characteristic sequences from a behavior e_j^i , where i denotes the pod and j represents the type of operation, a comparative analysis is conducted between e_j^i and each element in the sequence set of group n , symbolized as w_t^n . This process aims to determine the Longest Common Subsequence (Bergroth et al. 2000) s , as described in Eq. (2). Concurrently, we introduce a constant k and a coefficient r to establish the minimum valid sequence length ms , when the length of $f(e_j^i, w_t^n)$ falls below the minimum required length ms , e_j^i is directly integrated as a feature into the set w_t^n instead of $f(e_j^i, w_t^n)$, thereby mitigating the extraction of ineffectually brief subsequences, as delineated in Eq. (3).

$$s = \begin{cases} f(e_j^i, w_t^n), & \text{if } f(e_j^i, w_t^n) \geq ms \\ e_j^i, & \text{otherwise} \end{cases} \quad (1)$$

$$f(e_j^i, w_t^n) = \max(\{LCS(e_j^i, seq) | seq \in w_t^n\}) \quad (2)$$

$$ms = \begin{cases} |e_j^i| * r, & \text{if } |e_j^i| \cdot r \geq k \\ k, & \text{otherwise} \end{cases} \quad (3)$$

This algorithm for extracting feature sequences is designed to effectively summarize the normal behaviors of groups. While it may not match the precision achieved by machine learning or deep learning techniques, particularly with large datasets, it excels in scenarios characterized by small sample sizes or sparse pod behaviors-conditions commonly encountered in real production environments.

Algorithm 1 outlines the methodological steps for generating a behavioral baseline. This baseline is critical for underpinning subsequent efforts in anomaly detection.

Algorithm 1 Algorithm for behavior baseline generation

```

Input :  $G = \{g_n | n \text{ is group name}\}, g_n = \{e_j^i | i \text{ is pod name, } j \text{ is the type of operation}\},$ 
Output:  $\{W_n | n \text{ is group name}\}$ 
1 for  $g_n$  in  $G$  do
2    $w_{read}^n, w_{write}^n, w_{exec}^n, w_{conn}^n \leftarrow \{\}$ 
3   for  $e_j^i$  in  $g_n$  do
4     if  $j \in \{write, read, exec\}$  then
5        $seq, pattern \leftarrow findMaxLCS(e_j^i, w_t^n)$ 
6       if  $length \text{ of } seq < k$  then
7          $w_t^n \leftarrow append(w_t^n, obj)$ 
8       else
9          $w_t^n \leftarrow replacePattern(w_t^n, pattern, seq)$ 
10      end
11    else
12      if  $j = conn$  then
13         $g_{dst} \leftarrow findGroupByIP(e_j^i, G)$ 
14        if  $g_{dst} \neq nil$  then
15           $w_{conn}^n \leftarrow append(w_{conn}^n, g_{dst})$ 
16        else
17           $w_{conn}^n \leftarrow append(w_{conn}^n, ip)$ 
18        end
19      end
20    end
21  end
22   $W_n \leftarrow \{w_{read}^n, w_{write}^n, w_{exec}^n, w_{conn}^n\}$ 
23 end

```

Online detection

In the Online Detection phase, *ShadowKube* utilizes a sophisticated time window algorithm to continuously analyze cluster behaviors. By calculating a cumulative suspicion score for each time window, it effectively identifies deviations from established behavioral baselines, highlighting potential security threats or anomalies within the Kubernetes cluster environment.

Specifically, upon receiving behavior e_a^b , the *ShadowKube* identifies the corresponding group p via labels of e_a^b . If the group is unidentifiable, the suspicion score d increments by a fixed value x . Otherwise, depending on the type of operation a and the baseline w_a^p , different methodologies are applied to calculate d . The types of operations we focus on form the basis of various categories of malicious actions, including container escape, unauthorized file reading, server-side request forgery (SSRF), and arbitrary code execution. These fundamental operations can be further detailed as follows.

- **File Access.** *ShadowKube* iterates over the sequences in w_a^p , calculating the deviation of e_a^b from each

sequence and selecting the minimal deviation as the suspicion score d .

- **Command Execution.** *ShadowKube* identifies the first binary program in w_a^p that shares a feature sequence array with e_a^b , calculates the deviation between e_a^b and each feature sequence in the array, and adopts the minimal deviation as the suspicion score d .
- **Network Connection.** If an element in w_{conn}^p corresponds to a group, to which e_a^b belongs, or e_a^b is equal to that element, *ShadowKube* sets the suspicion score d to 0, otherwise a constant value c .

The deviation is assessed by quantifying the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform the feature sequence s into e_a^b using the Levenshtein distance algorithm (2002) and normalizing the result as a suspicion score, as described in Eq. (4).

For example, consider the anomaly detection process illustrated in Fig. 4, where the suspicious feature */EVIL* is compared to the baseline feature */NORMAL* to compute their difference. Transforming */EVIL* to */NORMAL* involves replacing “EVI” with “NOR” and inserting “MA”, resulting in a Levenshtein distance of 5. The normalized distance is therefore 0.714.

This method of calculating suspicion scores effectively reflects the degree of difference between observed behavior and the baseline. If the cumulative suspicion score D for window T_i exceeds the threshold L , it indicates abnormal behavior within that timeframe; otherwise, D is reset for the next window T_{i+1} . The algorithm for anomaly detection is detailed in Algorithm 2.

Addressing the **efficiency challenge** outlined before, our methodology quantitatively assesses the disparity between observed behavior and the established baseline with a computational complexity of $O(n^2)$. This efficiency ensures minimal computational overhead, making the approach particularly suitable for scenarios that demand rapid responsiveness within seconds.

$$d = \min\left(\left\{\frac{\text{lev}(e_a^b, s)}{|s|} \mid s \in w_a^p, b \in p\right\}\right) \quad (4)$$

Upon detecting abnormal behavior, *ShadowKube* relays this information to the actuator, which then formulates a defense strategy based on the current state of the cluster.

Algorithm 2 Algorithm for abnormal detecting

```

Input :  $\{W_n \mid n \text{ is group name}\}, e_j^i, D, c, x, L$ 
Output: Result
1 if  $e_j^i$  happened in next Time window then
2    $D \leftarrow 0$ 
3 end
4 if  $\text{getGroupName}(e_j^i) = \text{nil}$  then
5    $D \leftarrow D + x$ 
6 else
7    $\text{baseline} \leftarrow \text{GetBaselineByGroup}(w_j^i, g)$ 
8   if  $j \in \{\text{write}, \text{read}, \text{exec}\}$  then
9      $\text{suspicious} \leftarrow 0$ 
10    for  $s$  in baseline do
11       $\text{deviation} \leftarrow \text{lev}(s, e_j^i) / |s|$ 
12      if  $\text{deviation} < \text{suspicious}$  or  $\text{suspicious} = 0$  then
13         $\text{suspicious} \leftarrow \text{deviation}$ 
14      end
15    end
16     $D \leftarrow D + \text{suspicious}$ 
17  else
18     $\text{flag} \leftarrow \text{false}$ 
19    for  $\text{net}$  in baseline do
20      if ( $\text{net}$  is group and  $e_j^i \in \text{net}$ ) or ( $e_j^i = \text{net}$ ) then
21         $\text{flag} \leftarrow \text{true}$ 
22      end
23    end
24     $D \leftarrow \text{if } \text{flag} = \text{false} \text{ then } D + c \text{ else } D$ 
25  end
26 end
27  $\text{Result} \leftarrow \text{if } D > L \text{ then Abnormal else Normal}$ 

```

Strategy selection

Following the detection of anomalies, *ShadowKube* proceeds to the Strategy Selection phase, where it devises appropriate countermeasures based on the nature of the anomaly and the overall state of the production cluster. This may involve either transforming affected nodes and pods into honeypots, or direct elimination of pods, both of which aimed at maintaining operational integrity while mitigating security risks. The transformation to honeypots is prioritized by *ShadowKube* provided it does not impede the functionality of the production cluster, or the impact can be automatically repaired by the cluster thanks to the self-healing characteristic of Kubernetes. Otherwise, the actuator opts for alternative measures. The core idea of strategy selection is as follows.

Consider a cluster comprising a set of service resources, denoted as $\{S_i \mid i = 1, 2, 3, \dots, n\}$, where each service S_i encompasses a specific set of pods $\{P_1^i, P_2^i, \dots, P_k^i\}$, which are distributed across m nodes within the cluster, represented by $\{N_j \mid j = 1, 2, 3, \dots, m\}$. Each N_j is associated with persistent storage data, D_j . *ShadowKube* determines

the impact of migrating a target node N_x on the overall cluster availability under the following conditions:

- The persistent storage data D_j on the node is absent.
- The count of operational nodes exceeds a predetermined threshold, ensuring that the migration strategy does not exhaust the resources of the production cluster.
- There exists a set of replicas $\{r_1, r_2, \dots, r_n\}$ such that pods sets $\{P_{r_1}^1, P_{r_2}^2, \dots, P_{r_n}^n\}$ are available on nodes $\{N_1, N_2, \dots, N_{x-1}, N_{x+1}, \dots, N_m\}$.

Kubernetes' inherent load balancing and self-healing capabilities are designed to maximize application availability within the cluster. However, should the array of pods on alternative nodes fall short of providing a full suite of services, the application might temporarily become unavailable during migration, awaiting the re-establishment of missing pods on replacement nodes. The feasibility of adopting a migration strategy in these scenarios is contingent upon the presence of schedulable nodes within the cluster, a pivotal factor for the reconstitution of the absent pods.

Maliciousness handling

Upon strategy selection, *ShadowKube* initiates specific operations within the cluster. For instance, if the strategy involves deleting a particular pod, *ShadowKube* executes this action and subsequently informs the API Server to recreate the deleted pod. Similarly, when the strategy entails transforming the targeted node and pod into a honeypot, *ShadowKube* undertakes a sequence of operations on the target node, integrating it into a pre-established shadow cluster.

To effectively engage attackers and mitigate the impact radius discreetly and promptly, as delineated in the **efficiency challenge**, we propose a nuanced strategy that departs from traditional honeypot deployment practices. While static honeypots present a lower risk of detection by attackers due to their fixed nature, they often fall short in luring sophisticated threats. In contrast, strategies involving dynamic relocation, such as Live Migration, pose a risk of exposing the defensive mechanism through noticeable disturbances, including interruptions in shell connectivity. Consequently, to overcome the **technical challenge** that reduce attackers' perception, our method prioritizes the seamless in situ conversion of existing nodes and pods into honeypots, aiming to maintain attacker engagement without any apparent interruptions.

Nevertheless, the in situ conversion of a pod may lead to a scenario where the solitary node is disconnecting from its control plane, potentially arousing suspicion. To mitigate this concern, we have devised a shadow cluster

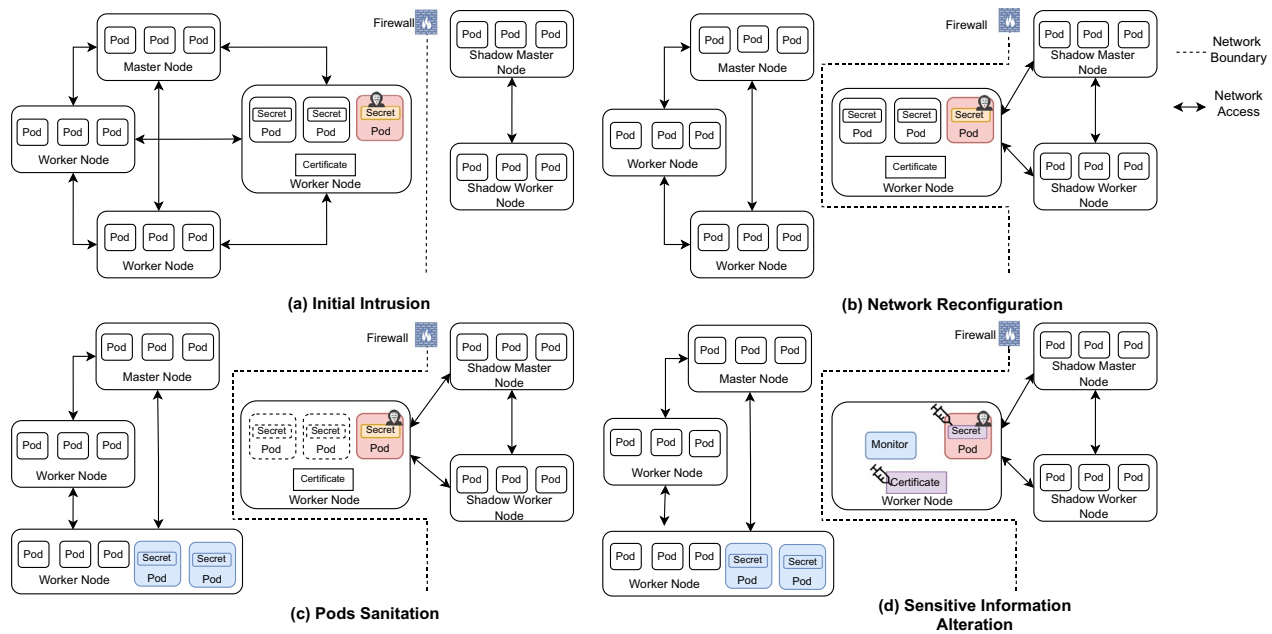


Fig. 5 A Transformation Process Example, where Initial Intrusion is detected and migrated by *ShadowKube* through three phases: Network Reconfiguration, Pod Sanitation, and Sensitive Information Alteration

Table 2 Misconfigurations in ShadowKube components

Component	Description	Impact	Exposed port
API server	Configured <i>insecure-port</i> , allowing access without authentication or authorization.	Attackers can compromise the cluster.	443, 6443, 8080
Kubelet	Enabled <i>anonymous</i> and configured <i>authorization.mode</i> , permitting unrestricted access to Kubelet.	Attackers can compromise the node.	10250, 10255
Docker daemon	Exposed remote API insecurely, allowing unauthorized access.	Attackers can execute malicious container operations.	2375
Kubernetes dashboard	Enabled <i>enable-skip-login</i> , allowing unauthorized access.	Attackers with Dashboard access can perform malicious operations on the cluster.	32000
Pods	mounted sensitive directory.	Attackers have access to host resources, allowing potential privilege escalation.	/
Pods	Enabled <i>privileged</i>	Attackers can perform privilege escalation.	/
Pods	Over-permissioned service accounts	Attackers can control additional resources even the cluster.	/

designed to dynamically integrate with the converted pods, thereby fabricating an ostensibly normal operational environment. Within this shadow cluster, services bearing identical names to those in the production environment are deployed to confuse the attacker, and commonly recognized insecure configurations are deliberately embedded within its components, as enumerated in Table 2. This strategy aims to maintain the illusion of a standard operational environment, reducing the likelihood of detection by the attacker.

The transformation process is executed in three meticulously planned phases: network reconfiguration, pod

sanitation, and sensitive information alteration. These steps are engineered to remain imperceptible to attackers, functioning beyond their immediate operational awareness. By maintaining the illusion of vulnerability, we stealthily hinder the attackers' ability to identify the honeypot, thereby increasing the likelihood of their prolonged engagement without detection. The transformation phases are illustrated as follows:

(1) Network Reconfiguration:

This phase aligns the pods on the target node with services within the shadow cluster, maintaining segregation between the production and shadow clusters. Network reconfiguration is achieved through *iptables*, redirecting ongoing processes on the target node to the shadow cluster upon API calls to production services, without alerting the attacker. This ensures that the attacker does not have access to other nodes in the production cluster, thus protecting all pods within them. This strategy enhances the decoy's credibility and potentially diverts attackers toward the shadow cluster during lateral movements.

(2) Pods Sanitation:

To protect additional pods on the target node from compromise, all pods, excluding the compromised one, are eliminated. This process remains invisible to the attacker due to the isolation between the container and host. The production cluster is then notified to recreate these pods on alternative nodes, thereby maintaining service continuity.

(3) Sensitive Information Alteration:

This step involves replacing sensitive information within the target node and pod, such as service account tokens and other Kubernetes credentials crucial for authenticating and authorizing access to the Kubernetes API in the production environment. These are replaced with valid credentials in the shadow cluster, making the replacement less likely to arouse suspicion from the attacker. Additionally, a monitor is installed on the target node to collect data originating from the attacker's behaviors.

Figure 5 illustrates the cluster's state through these phases. Thanks to the rapidity of situ conversion, the entire process can be completed in seconds, thus quickly controlling the attacker's range of activity. This strategy not only provides defenders with extra time to respond but also aids in gathering attack patterns, all while striving to reduce the intrusion's impact on the production cluster to the minimum extent feasible.

Ultimately, *ShadowKube* continuously monitors both the shadow cluster and the transformed node, ensuring the environment is reset when monitoring becomes non-beneficial, particularly if the attacker is no longer active. At the end of the trapping process, the actuator signals the traffic proxy to record the attacker and directs all subsequent traffic to the shadow cluster.

Evaluation

In this section, we outline the details of the experimental evaluation process, wherein we designed the following experiments to demonstrate the effectiveness of *ShadowKube*:

- **Vulnerability Exploitation Test.** To assess *ShadowKube*'s defensive capabilities against application vulnerabilities, we deployed applications with severe CVE vulnerabilities within the cluster and evaluated the system's ability to detect and redirect exploitation activities.
- **Misconfiguration Exploitation Test.** We deployed a simple microservice test application within the cluster, incorporating common misconfigurations typical in cloud-native scenarios. The effectiveness of *ShadowKube* in defending against malicious activities exploiting the test application and misconfigurations was then assessed.
- **Open Experiment.** To further demonstrate the effectiveness of *ShadowKube*'s entrapment capabilities, we exposed misconfigured clusters and microservice test applications to the public network. The attacks captured were analyzed and discussed.
- **Performance Test.** We conducted benchmark tests to evaluate the responsiveness of *ShadowKube* to malicious behaviour.

For comparison, we utilized Falco (2024), an open-source cloud-native runtime security tool that is widely used for threat detection Kubernetes environments, in vulnerability exploitation test and misconfiguration exploitation test.

Experiment environment

Ethical network isolation

The introduction of vulnerabilities within *ShadowKube* poses a potential risk wherein an attacker could compromise the entire cluster, thereby gaining control over all constituent machines. Given that attackers, upon accessing a single machine, are likely to attempt the compromise of additional machines within the network to expand the blast radius, all machines in the shadow cluster's network are consequently exposed to potential threats. Due to ethical considerations, we partitioned the experimental environment into two distinct segments. The first segment, utilized for evaluations excluding open experiments, operates within a Local Area Network (LAN) environment with no access to the public network. The second segment comprises an external-facing cluster environment. However, for ethical considerations, we did not deploy the cluster directly on public cloud infrastructure. Instead, a reverse proxy was deployed to

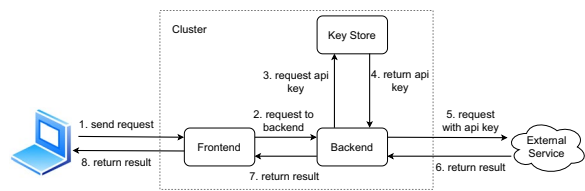


Fig. 6 Workflow of Test Application

expose service interfaces within the cluster, rendering this environment suitable for open experiments.

Details of test clusters

In each experiment, a simulated production cluster and the shadow cluster will be deployed across six local virtual machines, each equipped with a 2-core CPU and 4 GB of RAM. Four of these virtual machines will be configured to emulate a typical production environment, while the remaining two will be dedicated to forming the shadow cluster, serving defensive and entrapment roles. Additionally, we will utilize the snapshot mechanism of virtual machines to prevent irreversible damage to the experimental setup.

The testing clusters we employed were composed of Kubernetes 1.19.16 and Docker 24.0.5 on Debian system.

Parameters of Shadowkube

The configuration parameters of *ShadowKube* are detailed in Table 3. It is worth noting that through our repeated testing, a value of $k \geq 10$ helps avoid treating shallow system directories (e.g., /etc/) as feature subsequences. In addition, choosing r between 0.6 and 0.8 enables the extraction of more representative features while preventing overfitting. We also reasonably considered behaviors not belonging to any established group as malicious in the experiment and set the corresponding x and L values. Finally, to conserve physical resources, we restricted *ShadowKube* to performing honeypot conversion only when the number of nodes exceeds 3.

Experimental subjects

To emphasize the focus of each experiment, 43 severe vulnerabilities were selected from the CVE database,

spanning three categories of threats across 25 different open-source applications for vulnerability exploitation test.

As for other metrics, evaluations were carried out using an internally developed vulnerable microservice test application designed to simulate production-ready containerized applications. This application, structured as a basic weather query service, is depicted in Fig. 6, which starts with the front-end receiving user parameters to initiate a weather query request. Subsequently, the back-end retrieves the API key from storage, invokes an external weather API, and returns the results for display. As part of our experimental design, we deliberately introduced a code injection vulnerability into the front-end to establish an attack surface for exploitation testing.

Third-party expert team

Finally, we invited 30 cybersecurity experts specializing in cloud-native security to simulate normal user workflow and network attack scenarios. These experts, representing various organizations, possess diverse professional skill levels. These experts were tasked with conducting vulnerability exploitation and misconfiguration exploitation, with the results used to evaluate *ShadowKube's* effectiveness in real-world scenarios.

Vulnerability exploitation test

To evaluate *ShadowKube's* efficacy in mitigating application vulnerabilities, we conducted a study on the selected vulnerabilities, which were deployed in both shadow and production clusters, where normal operational workloads were executed for 10 min to allow *ShadowKube* to establish baselines. Then, we destroy the pods of containerized applications manually and wait for the cluster to recreate them, which simulates the dynamic scenario in the Kubernetes environment. After that, we monitored the production cluster under regular workload conditions for 30 min to assess the occurrence of false positives (FP), followed by the exploitation of the identified vulnerabilities to examine *ShadowKube's* detection capabilities, noted as true positives (TP). Before each experiment, The cluster environments are reset to ensure independence between experiments.

Table 3 Parameters of ShadowKube

Parameter	Description	Value
k	Constant used to define the minimum valid subsequence length	10
r	Coefficient used to determine the minimum valid subsequence length	0.7
x	Increased suspicion score when no groups are found	1
L	Suspicion score thresholds for determining malicious behavior	1
n	Minimum number of nodes to be retained in production cluster	3

Table 4 Results of detecting CVE vulnerabilities exploits

Threat impact	CVE	Application	Version	ShadowKube		Falco	
				TP	FP	TP	FP
Remote Code Execution	CVE-2023-46604	Apache ActiveMQ	5.17.3	✓	×	×	×
Remote Code Execution	CVE-2021-25646	Apache Druid	0.20.0	✓	×	×	×
Remote Code Execution	CVE-2021-45232	Apache APISIX	2.9	✓	×	×	×
Remote Code Execution	CVE-2022-46169	Cacti	1.2.22	✓	×	✓	✓
Remote Code Execution	CVE-2021-26120	CMSMS	2.2.9.1	✓	×	✓	✓
Remote Code Execution	CVE-2023-26360	Adobe ColdFusion	2018.0.15	✓	×	×	×
Remote Code Execution	CVE-2021-26084	Atlassian Confluence	7.4.10	✓	×	×	×
Remote Code Execution	CVE-2022-26134	Atlassian Confluence	7.13.6	✓	×	×	×
Remote Code Execution	CVE-2023-22527	Atlassian Confluence	8.5.3	✓	×	×	×
Remote Code Execution	CVE-2022-24706	Apache CouchDB	3.2.1	✓	×	×	×
Remote Code Execution	CVE-2024-36401	GeoServer	2.23.2	✓	×	×	×
Remote Code Execution	CVE-2021-22205	Gitlab	13.10.1	✓	×	×	×
Remote Code Execution	CVE-2023-4450	JeecgBoot	3.5.3	✓	×	×	✓
Remote Code Execution	CVE-2021-3129	Laravel	8.4.2	×	×	×	×
Remote Code Execution	CVE-2021-44228	Apache Log4j2	2.14.1	✓	×	×	×
Remote Code Execution	CVE-2023-38646	Metabase	0.46.6	✓	×	✓	×
Remote Code Execution	CVE-2023-49070	Apache OfBiz	18.12.09	✓	×	×	×
Remote Code Execution	CVE-2023-51467	Apache OfBiz	18.12.10	✓	×	×	×
Remote Code Execution	CVE-2024-38856	Apache OfBiz	18.12.14	✓	×	×	×
Remote Code Execution	CVE-2022-4223	pgAdmin	6.16	✓	×	×	×
Remote Code Execution	CVE-2023-5002	pgAdmin	7.6	✓	×	×	×
Remote Code Execution	CVE-2022-0543	Redis	5.0.7	✓	×	✓	×
Remote Code Execution	CVE-2022-22947	Spring Cloud Gateway	3.1.0	✓	×	×	×
Remote Code Execution	CVE-2022-22963	Spring Cloud Function	3.2.2	✓	×	×	×
Remote Code Execution	CVE-2022-22965	Spring WebMVC	5.3.17	✓	×	×	×
Remote Code Execution	CVE-2023-21839	Oracle Weblogic	12.2.1.3-2018	✓	×	×	×
Remote Code Execution	CVE-2021-29505	XStream	1.4.16	✓	×	×	×
Arbitrary File Access	CVE-2022-41678	Apache ActiveMQ	5.17.3	✓	×	×	×
Arbitrary File Access	CVE-2021-43008	Adminer	4.6.2	✓	×	×	×
Arbitrary File Access	CVE-2021-43798	Grafana	8.2.6	✓	×	×	×
Arbitrary File Access	CVE-2021-41773	Apache HTTP Server	2.4.49	✓	×	×	×
Arbitrary File Access	CVE-2021-42013	Apache HTTP Server	2.4.50	✓	×	×	×
Arbitrary File Access	CVE-2022-44268	ImageMagick	7.1.0-49	✓	×	×	×
Arbitrary File Access	CVE-2024-23897	Jenkins	2.441	✓	×	×	×
Arbitrary File Access	CVE-2021-28164	Eclipse Jetty	9.4.37	✓	×	×	×
Arbitrary File Access	CVE-2021-28169	Eclipse Jetty	9.4.40	✓	×	×	×
Arbitrary File Access	CVE-2021-34429	Eclipse Jetty	9.4.40	✓	×	×	×
Arbitrary File Access	CVE-2023-38633	libsvg	2.50.7	✓	×	×	×
Arbitrary File Access	CVE-2021-41277	Metabase	0.40.4	✓	×	×	✓
Arbitrary File Access	CVE-2024-4956	Nexus Repository Manager	3.68.0	✓	×	✓	×
Server-side request forgery	CVE-2021-21311	Adminer	4.7.8	✓	×	×	×
Server-side request forgery	CVE-2021-40438	Apache HTTP Server	2.4.48	✓	×	×	×
Server-side request forgery	CVE-2024-45507	Apache OfBiz	18.12.15	✓	×	×	×

For comparative analysis, we utilized *Falco* with its default official rule set, following identical testing procedures. The outcomes are detailed in Table 4. Note that

for CVE-2021-3129, the payload used by penetration testers executed *phpinfo()*, which did not trigger any system-level behavior on the target and, therefore, was not

Table 5 Result of detecting and handling attack against misconfigured components and application

Attack behavior	Threat category	ShadowKube			Falco	
		Detect	Migrate	Convert	Detect	Migrate
Read sensitive file via Frontend service.	Initial Access,Credential Access	✓	✓	✓	×	×
Steal API Key from Key Store service via Frontend service.	Initial Access, Credential Access	✓	✓	✓	×	×
Get reverse shell via Frontend service.	Execution, Persistence	✓	✓	✓	×	×
Deploy malicious Pods via misconfigured dashboard.	Execution, Persistence, Privilege Escalation, Discovery	✓	✓	×	✓	×
Deploy malicious Pods via misconfigured API Server.	Execution, Persistence	✓	✓	×	✓	×
Execute command in Pods via misconfigured Kubelet.	Execution	✓	✓	✓	×	×
Deploy malicious containers via misconfigured Docker Daemon.	Execution, Persistence	✓	✓	×	✓	×
Reconnaissance after gaining access to Pods.	Defense Evasion,Credential Access	✓	✓	✓	×	×
Escalation privileged Pods.	Privilege Escalation,Lateral Movement	✓	✓	✓	×	×
Steal Secret with privileged service account in Pod.	Credential Access, Lateral Movement	✓	✓	✓	✓	×
Deploy malicious containers via mounted docker.sock in Pod.	Persistence,Lateral Movement	✓	✓	×	✓	×

detected by *ShadowKube*. When the payload was modified to attempt executing malicious commands, *ShadowKube* successfully detected and mitigated the attack. The results revealed that *ShadowKube* achieved a 97.7% true positive rate with zero false positives across all test cases. In comparison, *Falco* demonstrated an accuracy rate of 11.6% with a false positive rate of 9.3%. In addition, by converting affected pods and nodes into honeypots, *ShadowKube* successfully identified and mitigated vulnerabilities, demonstrating its defense capabilities against intrusion activities.

Misconfiguration exploitation test

To evaluate *ShadowKube*'s ability to identify and counteract attacks arising from misconfigured components and applications, we deliberately configured components and applications insecurely within the production cluster, as detailed in Table 2, and exposed them to experts for exploitation.

Table 5 presents the result of detection and migration by *ShadowKube*, along with that by *Falco* as comparison. These attacks encompass ten threat categories identified in the Kubernetes threat matrix (2021), including the exploitation of vulnerabilities within the test application and attacks targeting components of the production cluster. Certain attack scenarios require attackers to satisfy specific conditions (e.g., have permission to access containers), which will be detected by *ShadowKube*. To assess the effectiveness in these scenarios, we disabled *ShadowKube* until the attack prerequisites were met. Notably, when attackers deployed malicious pods, replacing sensitive information proved challenging due to *ShadowKube*'s inability to automatically discover

sensitive information within them. Consequently, instead of converting the containers, *ShadowKube* opted to terminate them to neutralize such threats. Typically, attackers were detected and neutralized by *ShadowKube* during the early phases of their campaigns, such as upon acquiring service account credentials. In contrast, *Falco*, though detected Pods deployment successfully, were unable to migrate those threats. The findings confirm that *ShadowKube* effectively detects and mitigates prevalent attack vectors within the cluster.

Open experiment

To evaluate the effectiveness of *ShadowKube* within public network, we deployed test applications in both a test cluster and a shadow cluster. These applications were made accessible online via a reverse proxy to attract potential attackers, conducted within the ethical parameters of the study period from January 1 to January 31, 2024. The experiment received a total of 3,067 requests originating from 580 distinct IP addresses. After excluding requests from search engine crawlers, 3,020 requests remained for analysis. Utilizing the GeoLite2 database, we examined the geographical origins of these IP addresses and identified the top 20 countries and regions based on request volume, as depicted in Fig. 7.

Upon analyzing the data, 635 instances of attack attempts were identified, with a substantial majority targeting insecure cluster components rather than the test applications, as illustrated in Fig. 8. This preference indicates that attackers exploit components such as the API Server and Docker, likely due to their elevated permissions and the ease of automating attacks. Throughout the experiment, *ShadowKube* intervened by converting nodes and pods on 23 occasions in response to these

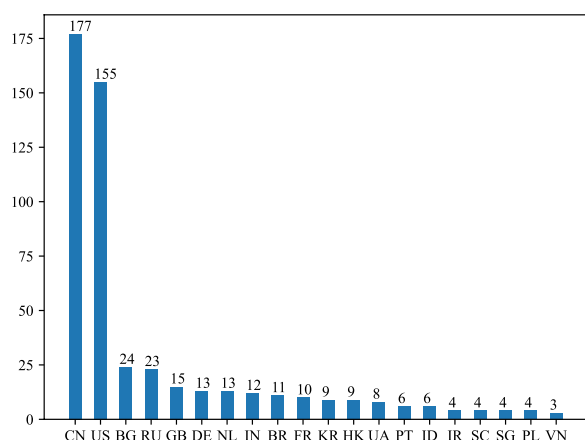


Fig. 7 Distribution of IP Location in Open Experiment

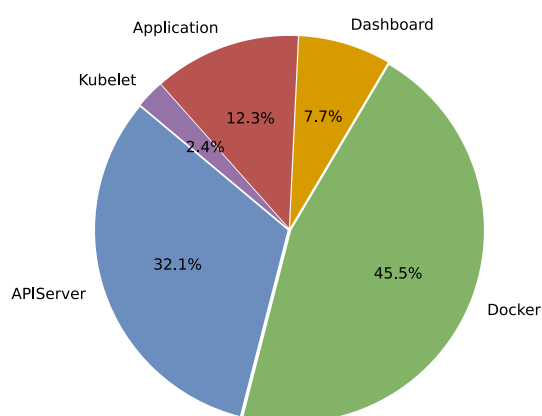


Fig. 8 Distribution of Attacked Components in Open Experiment

attacks, in addition to destroying malicious containers 48 times. These actions underscore the prevalence of automated threats and *ShadowKube*'s proactive defense measures. The traffic data associated with attack instances has been made available on our website (Chen 2024).

Further examination of the cluster logs revealed 6 malicious entities primarily engaged in cryptocurrency mining. These entities exhibited aggressive tactics, such as terminating other security services and processes to dominate the infected systems. Additionally, we identified 7 container escape attempts and 12 instances of attackers exploiting service account tokens to interact with the cluster's API Server. These findings highlight the sophisticated methods employed to compromise and exploit cluster resources.

Case study 1

On January 5, 2024, at 11:00 UTC, an IP address from Russia, 78.153.140.37, attempted to deploy a malicious container on a test cluster node via port 2375.

ShadowKube successfully removed the malicious container and redirected subsequent identical hostile requests to the shadow cluster, thereby isolating the threat. The attacker executed a command (`wget -q -O - 94.103.87.71/d.sh | sh`) that initiated complex, resource-intensive malicious activities, including the deletion of files in the `/tmp` directory to erase evidence. The attack involved terminating competing malicious containers and deploying the *kinsing* (2024) malware, which is designed for cryptocurrency mining. To ensure persistence, the attacker scheduled a crontab task for continuous script execution. After 20 min, *ShadowKube* reset the environment, mitigating the impact of the attack and blacklisting the offending IP address.

Case study 2

On January 27, 2024, at 23:45 UTC, an IP address originating from the United States, designated as 167.94.138.125, initiated a command injection attack against the test application, subsequently establishing a reverse shell. *ShadowKube* detected this intrusion and transformed the affected container's host node into a honeypot to mitigate the attack.

Subsequently, the attacker conducted reconnaissance within the compromised container and uploaded *cdk* (2024), a toolkit designed for container environment penetration, upon recognizing the Kubernetes context. Utilizing the service account token within the container, the attacker issued commands to the cluster's API Server with the aim of deploying a malicious daemonset resource to mine cryptocurrency using *xmrig* (2024). Due to the transformation initiated by *ShadowKube*, the hostile requests were rerouted to the shadow cluster, resulting in the malicious container operating within the shadow environment.

Approximately 20 min after the initial defensive response, *ShadowKube* reinstated the environment to its original state and blocked the malicious IP address, thereby concluding the containment and mitigation process.

Performance test

In this study, we examined the time required to migrate attack. Using the test application as a reference, we recorded the duration needed for a node hosting four pods to complete each phase of the transformation, repeating the experiment ten times. Our results indicate that, on average, the complete transformation was achieved in 9.612 s. The distribution of time across various stages is presented in Table 6, highlighting that the removal of non-invaded pods on the targeted node constitutes a significant portion of the total duration.

Table 6 Duration of each transition stage

Stage	Operation	Time used (s)
Attack detection	Detect abnormal behaviors.	1.03
Network reconfiguration	Modify <i>iptables</i> rules to redirect all requests from the compromised node to shadow cluster.	0.912
Network reconfiguration	Issue <i>iptables</i> rules to other nodes for isolating compromised one.	1.256
Pods sanitation	Remove other Pods located on the same node as the compromised Pod.	5.005
Sensitive information alteration	Replace sensitive information in the compromised node and Pod.	1.409

Notably, this duration is directly correlated with the number of containers present on the targeted node.

Upon activation of the transformation by *ShadowKube*, a network configuration phase is initiated, isolating the affected host and thereby preventing a cluster-wide attack within just 3.198 s. Furthermore, due to the independent nature of operations across different stages, there is potential to enhance efficiency by executing these operations asynchronously rather than sequentially. By adopting an asynchronous approach, we observed a substantial reduction in the average conversion time for the test application, decreasing to 5.39 s.

Considering that there are currently no tools with the same functionality for comparison, we cannot provide performance comparison data, but our experimental results show that the respond time of *ShadowKube* is acceptable while the average breakout time for interactive intrusion was more than several minutes (CrowdStrike 2024).

Furthermore, an analysis of *ShadowKube*'s performance overhead at various stages revealed that the baseline establishment process increased the average CPU usage per node by 1.3%, while baseline computation introduced an overhead of 3.7%. During the honeypot migration phase, the average CPU load on the affected node rose by 7.4%, primarily due to the container destruction operations performed during the transition. Given that the transition process lasts only a few seconds, this level of overhead is considered acceptable.

Discussion and limitation

Algorithm

Our algorithm for anomaly detection is deliberately designed to be simple and efficient to meet the real-time requirements of dynamic cloud-native environments. Complex algorithms, especially those integrating machine learning or deep learning techniques, often demand substantial computational resources and introduce processing delays, making them unsuitable for real-time detection scenarios. While such advanced methods might outperform in environments with large datasets, their utility diminishes in contexts

with limited data, which often occurs in dynamic and diverse container environments like Kubernetes. For example, CDL (2020), a machine learning-based anomaly detection solution for containerized environments, requires duplicating numerous instances of the target container application—a demand that is often impractical in actual production settings due to resource constraints. In contrast, our approach emphasizes real-time detection by accounting character features and behavior frequency. This design choice enhances detection accuracy without necessitating extensive computational resources or large datasets. By maintaining efficacy even with smaller datasets and reducing processing delays, our method is better suited for the variable landscape of cloud-native environments, ensuring timely and reliable anomaly detection.

Cost of baseline establishment

The accuracy with which the system's standard behavior is modeled is critical to the effectiveness of anomaly detection. To enhance the performance of *ShadowKube*, it is beneficial to collect extensive behavioral data during the baseline establishment phase. This process can be integrated with the testing phase following software development, as the behavioral data collected and analyzed can leverage the information generated during functional testing. Moreover, since this process can run in daemon mode, it introduces minimal additional complexity and does not significantly extend the overall testing duration. Finally, as demonstrated in performance test, the system overhead introduced by baseline establishment is relatively small and acceptable.

Cost of shadow cluster

The shadow cluster operates independently from the production environment, rendering its costs autonomous from those of the original cluster. *ShadowKube*'s architecture permits a minimal configuration comprising just one node, allowing for a significantly reduced cost footprint. Nevertheless, astute attackers may grow wary of a shadow cluster with too few nodes following a breach. To counteract this, defenders have the option to augment the

node count within the shadow cluster, thereby enhancing its capability to ensnare adversaries.

Limitation

In our experimental evaluation, we did not encounter any false positives when dealing with known vulnerabilities. However, we acknowledge that false positives may occur in real-world scenarios, especially under any of the following conditions, where the effectiveness of anomaly detection may be reduced. We considered the following three scenarios that could lead to false positives:

1) Similarity Between Malicious and Benign Behaviors: When malicious behaviors closely resemble benign container activities, the anomaly detection algorithm may fail to accurately capture the distinguishing characteristics of malicious actions.

2) Baseline Complexity in Dynamic Environments: In highly dynamic environments, the baseline can become overly complex. If the minimal feature behavior sequence within the baseline is too long, the resulting suspicion score during detection may be significantly weakened.

3) Evolution of Standard Behavior Sets: Over time, changes in the standard behavior of the dynamic environment may lead to an increase in false positives.

For scenarios 1), we propose continuous behavioral monitoring as a solution. An intrusion typically comprises a series of distinct malicious behaviors, so detecting any single malicious action can initiate a process to monitor subsequent behaviors for further indicators of compromise.

For scenarios 2), we believe that adopting the cloud-native development paradigm-decoupling program functions and distributing them across multiple containers-can mitigate baseline complexity. In cases where applications do not follow a microservices architecture, this issue may be alleviated by adjusting the normalization operations in the anomaly detection process, for example, by employing min-max scaling.

For **scenarios 3)**, two mitigation measures exist: the first is to periodically rebuild the behavior baseline, which is straightforward but requires balancing the trade-off between completely invalidating the baseline and affecting cluster availability; the second is to dynamically adjust thresholds based on the trend of suspicious score changes during the anomaly detection process.

Related work

Honeypot

Honeypots are proactive defense mechanisms designed to detect, warn, and counteract hacker intrusions, categorized by their interaction levels. Low-interaction honeypots, such as Honeyd (Honeyd 2024) or

Nepenthes (nepenthes 2024), simulate specific activities of services for automated attack detection, while high-interaction honeypots (López-Morales et al. 2020; Yang et al. 2023; Guan et al. 2023; Bringer et al. 2012) create realistic environments to collect comprehensive attack data.

In Kubernetes scenarios, existing studies related to honeypots can be categorized into two groups: those that focus on the management of resources in Kubernetes, and those that focus on the security of the Kubernetes platform itself. Gupta et al. (2023) proposed HoneyKube, a novel honeypot design that leverages the microservices architecture on Kubernetes; however, it is limited by its disregard for the inherent risks present in Kubernetes environments. Similarly, Kokolakis et al. (2022) developed HoneyChart, a framework that simplifies the deployment of honeypots on Kubernetes. Spahn et al. (2023) designed high-interaction honeypots based on Kubernetes and exposed them on public networks to reveal attacker activities, an approach that leans more toward investigative and research purposes rather than safeguarding Kubernetes. These studies do not address runtime security protection for container orchestration systems through the use of honeypots.

Inspired by Shadow Honeypots (Anagnostakis et al. 2005), which integrate honeypots with anomaly detection to process anomalous traffic and validate anomaly predictions-thereby isolating potential threats-we note that the anomaly detector in that work utilizes validation outcomes to enhance attack filtering, with any changes caused by traffic being handled or reverted based on intent. The primary limitation of Shadow Honeypots lies in the strong coupling between the honeypot and the actual operational processes, meaning that the honeypot program's state directly affects system stability. Furthermore, this approach is restricted to stateless binaries; for other system types, particularly Kubernetes workloads that depend on state data for managing operations, state rollback and reprocessing entail significantly greater complexity, thereby limiting its applicability in cloud-native scenarios.

In contrast, we design a novel honeypot capable of trapping and responding to attackers in a highly interactive manner that can be integrated into production Kubernetes environments to provide runtime defense. By isolating suspicious containers and nodes within the production environment and reconfiguring them into honeypots, *ShadowKube* implements a dynamic and transparent attacker entrapment technique to enhance Kubernetes security.

Kubernetes security

The widespread adoption of Kubernetes has highlighted significant security concerns, including misconfigurations and vulnerabilities that compromise cluster security (Rahman et al. 2023; Zeng et al. 2023; Spahn et al. 2023). To mitigate these risks, the industry has developed several open-source tools (Elkeid 2024; Kubescape 2024; Shopify 2024; KubeArmor 2024). Elkeid (2024) analyzes Kubernetes audit logs for intrusion detection and risk assessment, while Kubescape (2024) and Kubeaudit (2024) perform static analysis of system configurations. KubeArmor (2024) enforces security policies using the Linux Security Module (LSM). These tools rely on complex and elaborate policy rules, which can become burdensome as the rule set expands.

Academically, machine learning approaches such as those by Karn et al. (Karn et al. 2020), CDL (Lin et al. 2020), and KubAnomaly (Tien et al. 2019) have been proposed to detect malicious activities within Kubernetes clusters. CDL integrates online application classification and anomaly detection to handle diverse behaviors, while KubAnomaly employs neural networks to monitor and detect anomalous behaviors, including web service and CVE attacks. However, these methods rely on extensive behavioral data, presenting challenges in promptly responding to malicious actions.

In contrast, we propose an algorithm that establishes behavioral baselines and utilizes a trapping mechanism for Kubernetes defense, enabling swift detection and mitigation of risks in cloud-native environments by redirecting threats to a shadow cluster.

Conclusion

This paper proposes a Kubernetes security defense approach combining behavior analysis and dynamic honeypots-*ShadowKube*. The method establishes a behavioral baseline and measures deviations from it to assess the degree of abnormal behavior. Upon detecting anomalous behavior, *ShadowKube* can swiftly transform affected nodes and containers into honeypots in a covert manner, diverting threats to a shadow honeynet. Our experiments validate the efficacy of *ShadowKube* in vulnerability detection, rapid isolation, and node conversion, demonstrating its potential as a key tool for enhancing Kubernetes security and defending against evolving threats.

In future work, we aim to enhance *ShadowKube* by leveraging large language models to develop effective mechanisms for handling potential false positives, providing fine-grained feedback to improve the attack detection engine. Furthermore, to increase the obfuscation capabilities of *ShadowKube*, we encourage developers to expand the number of nodes in the shadow cluster at

the expense of some hardware resources. In addition, we plan to investigate the use of communication simulation techniques to mislead attackers, allowing us to simulate a large cluster size with minimal hardware resources.

Acknowledgements

The authors of the paper sincerely appreciate anonymous reviewers who reviewed this manuscript and provided constructive comments.

Author contributions

Qingwang Chen participated in all the work, designed the framework and experiments, and wrote the manuscript. Yuling Liu provided suggestions on the framework design and joined the discussion of this work. Ru Tan provided the suggestion of experiment. Ze Jin reviewed the manuscript and gave suggestions on the revision of the details of the article. Juxin Xiao modified and replicated the algorithm in a comparative experiment. Xiao Wang provided the suggestion of honeypots and the details of migration. Fangjiao Zhang provided the instruments of the framework. Qixu Liu provided suggestions on the framework design and joined the discussion of this work. All authors read and approved the final manuscript.

Funding

This work is supported by National Natural Science Foundation of China (Grant No.92467201), the Youth Innovation Promotion Association CAS (Grant No.Y2023047), the Science and Technology Project of State Grid (No. 5700-202352606A-3-2-ZN), the Key Laboratory of Network Assessment Technology at Chinese Academy of Sciences, and Beijing Key Laboratory of Network Security and Protection Technology.

Availability of data and materials

The split dataset used in this paper and the API list of our method are publicly available and could be accessed at: <https://sites.google.com/view/shadowkubernetes>.

Declarations

Competing interests

The authors declare that they have no Conflict of interest.

Received: 15 November 2024 Accepted: 23 January 2025

Published online: 25 August 2025

References

- Amazon (2024) Amazon web service. <https://aws.amazon.com/>
- Anagnostakis KG, Sidiroglou S, Akritidis P, Xinidis K, Markatos E, Keromytis AD (2005) Detecting targeted attacks using shadow honeypots. In: 14th USENIX Security Symposium (USENIX Security 05), USENIX Association, Baltimore, MD, <https://www.usenix.org/conference/14th-usenix-security-symposium/detecting-targeted-attacks-using-shadow-honeypots>
- Aquasec (2024) Container escape. <https://www.aquasec.com/cloud-native-academy/container-security/container-escape/>
- Aquasec (2024) Tracee. <https://github.com/aquasecurity/tracee>
- Bergroth L, Hakonen H, Raita T (2000) A survey of longest common subsequence algorithms. In: Proceedings Seventh International Symposium on String Processing and Information Retrieval. SPIRE 2000, 39–48, <https://doi.org/10.1109/SPIRE.2000.878178>
- Blackberry (2024) Falco bypasses. <https://github.com/blackberry/Falco-bypasses>
- Box HT (2024) Penetration testing reports. <https://www.hackthebox.com/blog/penetration-testing-reports-template-and-guide>
- Bringer ML, Chelmecki CA, Fujinoki H (2012) A survey: recent advances and future trends in honeypot research. Int J Comput Netw Inf Secur 4(10):63
- Canonical (2024) Canonical kubernetes usage report 2023. <https://juju.is/cloud-native-kubernetes-usage-report-2022>
- CDK (2024) Cdk. <https://github.com/cdk-team/CDK>

- Chen (2024) Shadowkube supporting materials. <https://sites.google.com/view/shadowkube>
- CrowdStrike (2024) Crowdstrike 2024 global threat report. <https://www.crowdstrike.com/global-threat-report/>
- Docker (2024a) Docker. <https://www.docker.com/>
- Docker (2024b) Docker swarm. <https://docs.docker.com/engine/swarm/>
- Elkeid (2024) Elkeid. <https://elkeid.bytedance.com/docs/>
- Falco (2024) Falco. <https://github.com/falcosecurity/falco>
- Google (2024) Google cloud. <https://cloud.google.com/>
- Guan C, Liu H, Cao G, Zhu S, Porta TFL (2023) Honeyiot: adaptive high-interaction honeypot for iot devices through reinforcement learning. Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks <https://api.semanticscholar.org/CorpusID:258182800>
- Gupta C, van Ede T, Continella A (2023) Honeykub: designing and deploying a microservices-based web honeypot. In: SecWeb 2023
- Honeyd (2024) Honeyd. <https://www.honeyd.org/>
- Karn RR, Kudva P, Huang H, Suneja S, Elfadel IM (2020) Cryptomining detection in container clouds using system calls and explainable machine learning. *IEEE Trans Parallel Distrib Syst* 32(3):674–691
- Katchinskiy M, Morag A (2023) leveraging kubernetes rbac to backdoor clusters. <https://www.aquasec.com/blog/leveraging-kubernetes-rbac-to-backdoor-clusters/>
- Kokolakis G, Ntousakis G, Karatsoris I, Antonatos S, Athanatos M, Ioannidis S (2022) Honeychart: automated honeypot management over kubernetes. In: European Symposium on Research in Computer Security, Springer, pp 321–328
- KubeArmor (2024) Kubearmor. <https://github.com/kubearmor/KubeArmor>
- Kubernetes (2024a) kube-apiserver. <https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>
- Kubernetes (2024b) Kubernetes. <https://kubernetes.io/>
- Kubernetes (2024c) Kubernetes pod. <https://kubernetes.io/docs/concepts/workloads/pods/>
- Kubernetes (2024d) Network policy. <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Kubernetes (2024e) Persistent volumes. <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>
- Kubernetes (2024f) Rbac authorization. <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- Kubescape (2024) Kubescape. <https://kubescape.io/>
- Lin Y, Tunde-Onadele O, Gu X (2020) Cdl: classified distributed learning for detecting security attacks in containerized applications. In: Proceedings of the 36th Annual Computer Security Applications Conference, Association for Computing Machinery, New York, NY, USA, ACSAC '20, p 179–188, <https://doi.org/10.1145/3427228.3427236>, <https://doi.org/10.1145/3427228.3427236>
- López-Morales E, Rubio-Medrano C, Doupé A, Shoshitaishvili Y, Wang R, Bao T, Ahn GJ (2020) Honeyplc: a next-generation honeypot for industrial control systems. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, pp 279–291
- Microsoft (2021) Threat matrix for kubernetes. <https://microsoft.github.io/Threat-Matrix-for-Kubernetes/>
- Microsoft (2024a) Detecting cryptomining attacks in the wild. <https://sysdig.com/blog/detecting-cryptomining-attacks-in-the-wild/>
- Microsoft (2024b) Microsoft defender for kubernetes. <https://learn.microsoft.com/en-us/azure/defender-for-cloud/defender-for-kubernetes-introduction>
- MITRE (2024) Kinsing. <https://attack.mitre.org/software/S0599/>
- Nepenthes (2024) nepenthes. <https://github.com/jrwren/nepenthes>
- NIST (2020) Cve-2020-15257. <https://nvd.nist.gov/vuln/detail/CVE-2020-15257>
- OWASP (2024) Owasp top ten. <https://owasp.org/www-project-top-ten/>
- Podman (2024) Podman. <https://podman.io/>
- Rahman A, Shamim SI, Bose DB, Pandita R (2023) Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology* 32(4):1–36
- Redhat (2023a) 2023 kubernetes security report. <https://www.redhat.com/en/resources/state-kubernetes-security-report-2023>
- Redhat (2023b) Kubernetes trends report 2023. <https://www.redhat.com/en/resources/kubernetes-adoption-security-market-trends-overview>
- Redhat (2024) Openshift. <https://www.openshift.com/>
- Redlock (2024) Tesla kubernetes hacked for cryptojacking. <https://www.bbc.com/news/technology-43140005>
- Schulz KU, Mihov S (2002) Fast string correction with levenshtein automata. *Int J Doc Anal Recogn* 5:67–85
- Shopify (2024) kubeaudit. <https://github.com/Shopify/kubeaudit>
- Spahn N, Hanke N, Holz T, Kruegel C, Vigna G (2023) Container orchestration honeypot: Observing attacks in the wild. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, pp 381–396
- spectrocloud (2023) 2023 state of production kubernetes. <https://info.spectrocloud.com/report-kubernetes-2023>
- Tetragon (2024) Tetragon. <https://tetragon.io/>
- thenewstack (2023) State of kubernetes in production. <https://thenewstack.io/the-2023-state-of-kubernetes-in-production/>
- Tien CW, Huang TY, Tien CW, Huang TC, Kuo SY (2019) Kubanomaly: Anomaly detection for the docker orchestration platform with neural network approaches. *Engineering reports* 1(5):e12,080
- VMware (2023) State of kubernetes. <https://go-vmware.broadcom.com/register-the-state-of-kubernetes>
- Vulhub (2024) Vulhub. <https://github.com/vulhub/vulhub>
- wikipedia (2024a) Lateral movement. <https://www.cloudflare.com/learning/security/glossary/what-is-lateral-movement/>
- wikipedia (2024b) Privilege escalation. <https://bishopfox.com/blog/kubernetes-pod-privilege-escalation>
- Wiz (2024) Kubernetes security report 2023. <https://www.wiz.io/lp/the-2023-kubernetes-security-report>
- Xmrig (2024) xmrig. <https://github.com/xmrig/xmrig>
- Yang X, Yuan J, Yang H, Kong Y, Zhang H, Zhao J (2023) A highly interactive honeypot-based approach to network threat management. *Future Internet* 15:127, <https://api.semanticscholar.org/CorpusID:257827901>
- Zeng Q, Kavousi M, Luo Y, Jin L, Chen Y (2023) Full-stack vulnerability analysis of the cloud-native platform. *Comput & Secur* 129(103):173

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.